

MODUL PRAKTIKUM

Virtual Networking, NFV & SDN

dalam Lingkungan Virtualisasi

Mata Kuliah Virtualisasi & Keamanan Jaringan	Program Studi S1 Teknik Informatika
Durasi 2 Pertemuan x 1,5 Jam	Tools VirtualBox MikroTik CHR Ubuntu 24.04

PENDAHULUAN

A. Deskripsi Modul

Modul praktikum ini dirancang untuk memberikan pengalaman langsung (hands-on) kepada mahasiswa dalam mengimplementasikan konsep Virtual Networking, Network Function Virtualization (NFV), dan Software Defined Networking (SDN) menggunakan perangkat lunak open-source dan image router virtual.

Melalui dua sesi praktikum, mahasiswa akan membangun topologi jaringan virtual dari awal, mengkonfigurasi router virtual MikroTik CHR sebagai contoh implementasi NFV, mengeksplorasi Open vSwitch sebagai komponen SDN, serta memahami mekanisme isolasi jaringan container menggunakan Docker.

B. Capaian Pembelajaran Praktikum

Setelah menyelesaikan seluruh kegiatan praktikum ini, mahasiswa diharapkan mampu:

- Menjelaskan dan membedakan mode jaringan pada hypervisor VirtualBox (NAT, Bridged, Internal Network, Host-Only, NAT Network).
- Membangun topologi jaringan virtual multi-segmen menggunakan VirtualBox dan MikroTik CHR.
- Mengkonfigurasi layanan DHCP Server, static routing, dan NAT Masquerade pada MikroTik CHR sebagai implementasi NFV.
- Menginstall dan mengkonfigurasi Open vSwitch (OVS) pada Ubuntu 24.04 sebagai virtual switch berbasis SDN.
- Membuat dan menginspeksi jaringan container menggunakan Docker, termasuk network isolation.
- Mengimplementasikan filter firewall pada MikroTik CHR untuk mengamankan segmen jaringan virtual.
- Mendokumentasikan dan menganalisis hasil konfigurasi jaringan virtual secara sistematis.

C. Alat dan Bahan

No.	Perangkat/Software	Keterangan
1	VirtualBox (versi terbaru)	Hypervisor type-2, gratis, tersedia di virtualbox.org
2	MikroTik CHR Image (.vdi)	Cloud Hosted Router, download di mikrotik.com/download
3	Ubuntu Server 24.04 ISO	Minimal server install, download di ubuntu.com
4	PC Host minimal 8GB RAM	Diperlukan untuk menjalankan 3 VM sekaligus
5	Koneksi Internet (saat lab)	Untuk download package di VM Ubuntu

D. Petunjuk Umum Pengerjaan

 **Baca sebelum memulai praktikum!**

- Baca setiap langkah dengan teliti sebelum menjalankan perintah.
- Ambil snapshot VirtualBox setelah setiap tahap lab berhasil sebagai titik restore.
- Catat setiap hasil output perintah pada Lembar Kerja (halaman terakhir modul).
- Jika terjadi kesalahan, jangan panik — baca pesan error dengan cermat, ulangi langkah dari snapshot terakhir.
- Tanda # pada awal baris perintah berarti komentar (tidak perlu diketik).
- Tanda \$ berarti perintah dijalankan sebagai user biasa, tanda # berarti sebagai root/sudo.

PRAKTIKUM 1

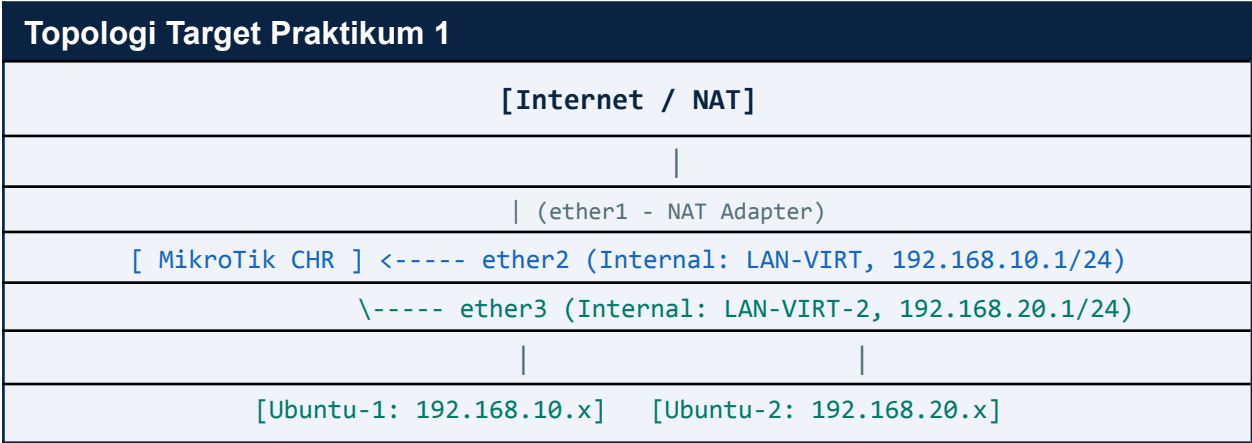
Virtual Networking Fundamentals, NFV dengan MikroTik CHR

Durasi: 1,5 Jam | Tools: VirtualBox + MikroTik CHR + Ubuntu 24.04

A. Tujuan Praktikum 1

- Memahami dan mengkonfigurasi mode jaringan VirtualBox.
- Mengimport dan mengkonfigurasi MikroTik CHR sebagai virtual router (contoh NFV).
- Membangun topologi jaringan virtual dua segmen (LAN-VIRT dan LAN-VIRT-2).
- Mengkonfigurasi DHCP Server dan NAT Masquerade pada CHR.
- Membuktikan routing antar segmen jaringan virtual.

B. Topologi Jaringan



Perangkat	Konfigurasi Jaringan
MikroTik CHR	ether1: DHCP (NAT ke internet) ether2: 192.168.10.1/24 ether3: 192.168.20.1/24
Ubuntu-1	IP: 192.168.10.x (DHCP dari CHR) Gateway: 192.168.10.1
Ubuntu-2	IP: 192.168.20.x (DHCP dari CHR) Gateway: 192.168.20.1

C. Langkah Praktikum

TAHAP 1	Persiapan dan Import MikroTik CHR ke VirtualBox
---------	---

Langkah pertama adalah mengimport image MikroTik CHR yang sudah berbentuk file .vdi (Virtual Disk Image) ke dalam VirtualBox sebagai VM baru.

- Buka VirtualBox, klik menu Machine > New.
- Isi nama VM: MikroTik-CHR | Type: Other | Version: Other/Unknown (64-bit).
- Alokasikan RAM: 256 MB (cukup untuk CHR).

4. Pada bagian Hard Disk, pilih Use an existing virtual hard disk file.
5. Klik ikon folder, browse ke file mikrotik-chr-x.xx.x.vdi yang sudah didownload.
6. Klik Create untuk menyelesaikan pembuatan VM.

Konfigurasi Network Adapter CHR

Klik kanan VM MikroTik-CHR > Settings > Network. Pastikan:

- Adapter 1 (ether1): Attached to NAT
- Adapter 2 (ether2): Attached to Internal Network, name: LAN-VIRT
- Adapter 3 (ether3): BELUM aktifkan dulu, akan dikonfigurasi di Tahap 4

7. Klik OK dan jalankan VM: klik Start (tombol hijau panah kanan).
8. Tunggu proses booting CHR selesai (muncul prompt login).
9. Login dengan username: admin dan password: (kosong, langsung Enter).

Verifikasi Tahap 1

Jika muncul MikroTik prompt dan login berhasil, lanjut ke Tahap 2.

TAHAP 2 Konfigurasi IP Address dan DHCP Server pada CHR

Pada tahap ini kita akan mengkonfigurasi alamat IP pada setiap interface CHR dan mengaktifkan layanan DHCP Server agar Ubuntu-1 mendapatkan IP secara otomatis.

2.1 Konfigurasi IP Address Interface

Ketik perintah berikut di konsol MikroTik CHR:

```
# Lihat daftar interface yang aktif
/interface print

# Pasang IP pada ether2 (LAN-VIRT, segmen 192.168.10.0/24)
/ip address add address=192.168.10.1/24 interface=ether2

# Verifikasi IP sudah terpasang
/ip address print
```

Output yang Diharapkan	Penjelasan
/ip address print menampilkan ether2 dengan 192.168.10.1/24	IP berhasil dipasang pada interface ether2
Status ether2: R (running)	Interface aktif dan siap menerima koneksi

2.2 Konfigurasi DHCP Server untuk Segmen LAN-VIRT

```
# Buat pool IP untuk DHCP (range 192.168.10.100 - 192.168.10.200)
```

```

/ip pool add name=dhcp-pool-1 ranges=192.168.10.100-192.168.10.200

# Buat DHCP Network (gateway dan DNS untuk klien)
/ip dhcp-server network add address=192.168.10.0/24 gateway=192.168.10.1
dns-server=8.8.8.8

# Aktifkan DHCP Server pada interface ether2
/ip dhcp-server add name=dhcp-ether2 interface=ether2 address-pool=dhcp-pool-1
disabled=no

# Verifikasi DHCP Server aktif
/ip dhcp-server print

```

2.3 Konfigurasi NAT Masquerade (Akses Internet)

Agar VM Ubuntu-1 bisa mengakses internet melalui CHR, kita perlu mengkonfigurasi NAT Masquerade pada interface WAN (ether1).

```

# Pastikan ether1 mendapat IP dari NAT VirtualBox
/ip dhcp-client print

# Tambahkan rule NAT Masquerade
/ip firewall nat add chain=srcnat out-interface=ether1 action=masquerade

# Verifikasi rule NAT
/ip firewall nat print

```

✓ Verifikasi Tahap 2

Pastikan /ip dhcp-server print menampilkan dhcp-ether2 dengan status running=yes

TAHAP 3 Setup VM Ubuntu-1 dan Verifikasi Konektivitas

Buat VM Ubuntu-1 yang akan terhubung ke segmen LAN-VIRT melalui Internal Network VirtualBox.

1. Buat VM baru di VirtualBox: Name: Ubuntu-1 | Type: Linux | Version: Ubuntu (64-bit).
2. RAM: 1024 MB, CPU: 1, Disk: 10 GB (dynamically allocated).
3. Settings > Network > Adapter 1: Internal Network > name: LAN-VIRT.
4. Pasang ISO Ubuntu 24.04 dan install minimal server (tanpa GUI).
5. Setelah install selesai, login dan jalankan perintah verifikasi:

```

# Cek status network interface
ip addr show

```

```
# Cek apakah DHCP berhasil mendapat IP dari CHR
# Harusnya mendapat IP di range 192.168.10.100-200
ip route show
```

```
# Test ping ke gateway (CHR)
ping -c 4 192.168.10.1
```

```
# Test ping ke internet melalui CHR
ping -c 4 8.8.8.8
```

```
# Test resolusi DNS
ping -c 4 google.com
```

Jika ping ke 8.8.8.8 berhasil, berarti NAT Masquerade pada CHR bekerja dengan benar. Ini membuktikan bahwa MikroTik CHR berfungsi sebagai Network Function Virtualization (NFV) — sebuah router fisik yang digantikan oleh VM.

TAHAP 4

Penambahan Segmen Kedua (LAN-VIRT-2) dan Routing Antar Segmen

Pada tahap ini kita akan menambahkan segmen jaringan kedua (LAN-VIRT-2) dan membuktikan bahwa routing antar segmen dapat dilakukan oleh CHR.

4.1 Tambah Adapter 3 pada CHR

1. Matikan VM MikroTik-CHR (shutdown dengan perintah `/system shutdown` dari konsol).
2. Settings > Network > Adapter 3: centang Enable, pilih Internal Network, name: LAN-VIRT-2.
3. Jalankan kembali VM MikroTik-CHR.

```
# Di konsol CHR setelah boot ulang
# Pasang IP pada ether3 (LAN-VIRT-2, segmen 192.168.20.0/24)
/ip address add address=192.168.20.1/24 interface=ether3

# Buat pool DHCP kedua
/ip pool add name=dhcp-pool-2 ranges=192.168.20.100-192.168.20.200

# Buat DHCP Network untuk segmen kedua
/ip dhcp-server network add address=192.168.20.0/24 gateway=192.168.20.1
dns-server=8.8.8.8
```

```
# Aktifkan DHCP Server pada ether3
/ip dhcp-server add name=dhcp-ether3 interface=ether3 address-pool=dhcp-pool-2
disabled=no

# Verifikasi semua konfigurasi
/ip address print
/ip dhcp-server print
```

4.2 Buat VM Ubuntu-2 di Segmen LAN-VIRT-2

1. Buat VM baru: Name: Ubuntu-2, spesifikasi sama dengan Ubuntu-1.
2. Settings > Network > Adapter 1: Internal Network > name: LAN-VIRT-2.
3. Install Ubuntu Server 24.04, login, dan verifikasi:

```
# Di Ubuntu-2: Verifikasi mendapat IP di segmen 192.168.20.x
ip addr show

# Test ping ke gateway Ubuntu-2 (CHR ether3)
ping -c 4 192.168.20.1

# TEST ROUTING ANTAR SEGMENT (kunci percobaan ini!)
# Ping dari Ubuntu-2 ke IP Ubuntu-1 (192.168.10.x)
ping -c 4 192.168.10.100

# Traceroute untuk melihat path routing melalui CHR
traceroute 192.168.10.100
```



Analisis Hasil Routing

Perhatikan output traceroute: hop pertama adalah 192.168.20.1 (CHR ether3). Ini membuktikan bahwa CHR berfungsi sebagai router yang meneruskan paket antar dua segmen jaringan yang berbeda. Dalam konteks NFV, fungsi routing ini yang biasanya dilakukan oleh hardware router fisik, kini dijalankan oleh VM (CHR).



AMBIL SNAPSHOT SEKARANG

Beri nama snapshot: Lab1-Selesai. Ini adalah titik restore jika ada masalah di praktikum selanjutnya.

D. Pertanyaan Analisis Praktikum 1



Jawab pertanyaan berikut berdasarkan hasil praktikum Anda

1. Sebutkan dan jelaskan perbedaan mode jaringan VirtualBox yang Anda gunakan dalam lab ini (NAT dan Internal Network). Mengapa ether1 menggunakan NAT sementara ether2 dan ether3 menggunakan Internal Network?

2. Jelaskan mengapa Ubuntu-2 (192.168.20.x) dapat melakukan ping ke Ubuntu-1 (192.168.10.x) padahal keduanya berada di Internal Network yang berbeda. Gambarkan alur paket ICMP secara lengkap!

3. Apa yang dimaksud dengan NFV dan bagaimana MikroTik CHR merepresentasikan konsep tersebut dalam lab ini?

PRAKTIKUM 2

SDN dengan Open vSwitch, Container Networking, dan Virtual Firewall

Durasi: 1,5 Jam | Tools: Ubuntu 24.04 + Open vSwitch + Docker + MikroTik CHR



Prasyarat

Praktikum 2 dilanjutkan dari hasil Praktikum 1. Restore snapshot Lab1-Selesai jika diperlukan. Pastikan VM Ubuntu-1, Ubuntu-2, dan MikroTik-CHR dalam keadaan aktif.

A. Tujuan Praktikum 2

1. Menginstall dan mengkonfigurasi Open vSwitch (OVS) pada VM Ubuntu sebagai implementasi SDN.
2. Memahami perbedaan antara Linux Bridge tradisional dan OVS.
3. Menginstall Docker dan mengeksplorasi mekanisme jaringan container.
4. Membuktikan isolasi jaringan antar Docker network yang berbeda.
5. Mengimplementasikan Virtual Firewall menggunakan filter rules MikroTik CHR.

B. Langkah Praktikum

TAHAP 5

Instalasi dan Eksplorasi Open vSwitch (OVS) di Ubuntu-1

Open vSwitch (OVS) adalah virtual switch open-source yang mendukung protokol OpenFlow, menjadikannya komponen utama dalam implementasi Software Defined Networking (SDN). Berbeda dengan Linux Bridge yang sederhana, OVS mendukung VLAN tagging, tunneling (VXLAN/GRE), dan kontrol terpusat via SDN controller.

5.1 Instalasi Open vSwitch

Jalankan perintah berikut di terminal Ubuntu-1:

```
# Update package list
sudo apt update

# Install Open vSwitch
sudo apt install -y openvswitch-switch

# Verifikasi instalasi dan status service
sudo systemctl status ovs-vswitchd

# Cek versi OVS
sudo ovs-vsctl --version
```

5.2 Membuat dan Mengeksplorasi OVS Bridge

```
# Buat virtual switch baru bernama 'ovs-br0'
sudo ovs-vsctl add-br ovs-br0

# Lihat seluruh topologi OVS (bridge, port, interface)
sudo ovs-vsctl show

# Lihat informasi detail bridge (termasuk OpenFlow info)
sudo ovs-ofctl show ovs-br0

# Bandingkan dengan Linux Bridge tradisional
ip link show type bridge
brctl show # perlu install bridge-utils jika belum ada
```

Aspek	Linux Bridge	Open vSwitch
Perintah konfigurasi	brctl / ip link	ovs-vsctl / ovs-ofctl
VLAN Support	Terbatas	Penuh (802.1Q)
OpenFlow Support	Tidak	Ya (SDN-ready)
Tunneling	Tidak	VXLAN, GRE, Geneve
Kontrol terpusat	Tidak	Ya (via Controller)

5.3 Konfigurasi VLAN Tagging pada OVS

Demonstrasi salah satu keunggulan OVS: VLAN tagging pada tingkat port.

```
# Tambahkan port (interface virtual) ke bridge OVS
sudo ip tuntap add tap0 mode tap
sudo ip link set tap0 up
sudo ovs-vsctl add-port ovs-br0 tap0

# Set VLAN tag 10 pada port tap0
sudo ovs-vsctl set port tap0 tag=10

# Verifikasi: tap0 sekarang berada di VLAN 10
sudo ovs-vsctl show

# Tambahkan port kedua di VLAN berbeda (20)
sudo ip tuntap add tap1 mode tap
sudo ip link set tap1 up
sudo ovs-vsctl add-port ovs-br0 tap1
```

```
sudo ovs-vsctl set port tap1 tag=20
```

```
# List semua port beserta VLAN tag-nya
```

```
sudo ovs-vsctl list port
```



Makna VLAN di OVS

tap0 (VLAN 10) dan tap1 (VLAN 20) berada dalam bridge yang sama, namun terisolasi satu sama lain pada layer 2. Traffic hanya dapat melewati batas VLAN jika ada routing di layer 3. Ini adalah prinsip network segmentation dalam SDN.



Verifikasi Tahap 5

Pastikan output 'sudo ovs-vsctl show' menampilkan bridge ovs-br0 dengan dua port (tap0 tag=10 dan tap1 tag=20).

TAHAP 6

Docker Container Networking

Docker mengimplementasikan jaringan container menggunakan mekanisme Linux Network Namespace yang menjadi fondasi isolasi jaringan. Setiap container mendapatkan network stack tersendiri (interface, routing table, iptables) yang terisolasi dari host dan container lain.

6.1 Instalasi Docker

```
# Install Docker menggunakan apt
```

```
sudo apt update
```

```
sudo apt install -y docker.io
```

```
# Aktifkan dan jalankan Docker service
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
# Tambahkan user ke grup docker agar tidak perlu sudo
```

```
sudo usermod -aG docker $USER
```

```
newgrp docker
```

```
# Verifikasi Docker berjalan
```

```
docker --version
```

```
docker info
```

6.2 Eksplorasi Network Default Docker

```
# Lihat daftar network yang tersedia di Docker
```

```
docker network ls
```

```
# Inspect detail network 'bridge' (default)
```

```
docker network inspect bridge

# Perhatikan subnet dan gateway docker0 bridge
# Default: 172.17.0.0/16, gateway 172.17.0.1

# Lihat interface docker0 dari sisi host
ip addr show docker0
ip link show type bridge
```

6.3 Membuat Container dan Menguji Konektivitas

```
# Jalankan container pertama secara background (-d) interaktif (-it)
docker run -dit --name ct1 ubuntu:24.04 bash

# Jalankan container kedua
docker run -dit --name ct2 ubuntu:24.04 bash

# Lihat IP address masing-masing container
docker inspect ct1 | grep IPAddress
docker inspect ct2 | grep IPAddress

# Install ping di dalam container ct1
docker exec ct1 apt update -q
docker exec ct1 apt install -y iputils-ping -q

# Test ping dari ct1 ke ct2 (dalam network bridge yang sama)
docker exec ct1 ping -c 4 <IP-ct2>

# Hasilnya: ping BERHASIL (satu network bridge)
```

6.4 Membuat Custom Network dan Membuktikan Isolasi

Ini adalah inti dari Docker networking: container dalam network yang berbeda tidak dapat saling berkomunikasi secara langsung.

```
# Buat custom Docker network baru
docker network create --subnet=172.20.0.0/16 --gateway=172.20.0.1 virt-net

# Verifikasi network baru terbuat
docker network ls
docker network inspect virt-net

# Jalankan container ketiga di dalam custom network
```

```

docker run -dit --name ct3 --network virt-net ubuntu:24.04 bash

# Lihat IP ct3 (harusnya di range 172.20.x.x)
docker inspect ct3 | grep IPAddress

# Install ping di ct3
docker exec ct3 apt update -q && docker exec ct3 apt install -y iputils-ping -q

# TEST ISOLASI: ping dari ct3 ke ct1/ct2 (berbeda network)
docker exec ct3 ping -c 4 <IP-ct1>

# Hasilnya: ping GAGAL (network terisolasi, tidak bisa reach)

# Bandingkan: ct1 ke ct2 masih bisa ping (satu network)
docker exec ct1 ping -c 4 <IP-ct2>

```

Skenario Ping	Hasil yang Diharapkan
ct1 -> ct2 (sama-sama di network bridge default)	BERHASIL - Satu network, satu subnet
ct3 -> ct1 (virt-net vs bridge default)	GAGAL - Berbeda network, terisolasi
ct3 -> ct2 (virt-net vs bridge default)	GAGAL - Berbeda network, terisolasi
ct3 -> 172.20.0.1 (gateway virt-net)	BERHASIL - Gateway dalam satu network



Mengapa Isolasi ini Penting?

Ini adalah implementasi prinsip SDN: network isolation memungkinkan segmentasi workload yang berbeda (misalnya: frontend, backend, database) agar tidak saling mengakses satu sama lain secara langsung, meningkatkan keamanan dan manageability.



AMBIL SNAPSHOT SEKARANG

Beri nama snapshot: Lab2-Docker-Selesai.

TAHAP 7

Virtual Firewall dengan MikroTik CHR

Pada tahap terakhir ini, kita akan mengimplementasikan Virtual Firewall menggunakan fitur Filter Rules pada MikroTik CHR. Ini adalah contoh nyata Network Function Virtualization (NFV) untuk fungsi keamanan: firewall hardware fisik digantikan oleh software firewall yang berjalan di atas VM.

7.1 Memahami Arsitektur Firewall CHR

MikroTik CHR menggunakan konsep chain dalam firewall:

- chain=input: Traffic yang masuk menuju router itu sendiri.
- chain=output: Traffic yang keluar dari router.
- chain=forward: Traffic yang melewati router (antar segmen).

Untuk memfilter traffic antar segmen jaringan (Ubuntu-1 dan Ubuntu-2), kita menggunakan chain=forward.

7.2 Implementasi Rule Blokir ICMP Antar Segmen

```
# Di konsol MikroTik CHR
# Lihat rule firewall yang sudah ada
/ip firewall filter print

# Tambahkan rule: blokir ICMP dari LAN-VIRT-2 ke LAN-VIRT
/ip firewall filter add chain=forward \
src-address=192.168.20.0/24 \
dst-address=192.168.10.0/24 \
protocol=icmp \
action=drop \
comment=BLOKIR-ICMP-VIRT2-KE-VIRT1

# Verifikasi rule terpasang
/ip firewall filter print
```

7.3 Pengujian Rule Firewall

Lakukan pengujian dari terminal Ubuntu-2:

```
# Di Ubuntu-2: Test ping ke Ubuntu-1 SETELAH rule firewall aktif
ping -c 4 192.168.10.100
# Hasilnya: 100% packet loss (ICMP terblokir oleh firewall)

# Perhatikan counter rule di CHR bertambah
# (kembali ke konsol CHR)
/ip firewall filter print stats
# Kolom 'packets' dan 'bytes' pada rule drop akan bertambah
```

7.4 Pengujian Selektivitas Firewall (Hanya ICMP yang Terblokir)

```
# Install netcat di Ubuntu-1 (untuk test TCP)
# Di Ubuntu-1:
sudo apt install -y ncat
ncat -l 8080 # Listen di port 8080

# Di Ubuntu-2: Test koneksi TCP ke Ubuntu-1 port 8080
```

```
# (ICMP terblokir tapi TCP masih bisa melewati)
ncat 192.168.10.100 8080
# Hasilnya: koneksi TCP BERHASIL
# Ini membuktikan firewall bekerja selektif per protokol
```

7.5 Menonaktifkan dan Mengaktifkan Rule

```
# Di CHR: Nonaktifkan rule firewall (nomor rule adalah 0 jika pertama)
/ip firewall filter disable 0

# Kembali ke Ubuntu-2: ping sekarang BERHASIL lagi
ping -c 4 192.168.10.100

# Aktifkan kembali rule firewall
/ip firewall filter enable 0

# Ping kembali GAGAL
# Eksperimen: ubah action dari 'drop' menjadi 'reject'
/ip firewall filter set 0 action=reject
# Apa perbedaan perilaku ping dengan action reject vs drop?
```



Drop vs Reject

action=drop: Paket dibuang diam-diam, pengirim tidak mendapat respons (timeout).
 action=reject: Pengirim mendapat pesan ICMP 'port unreachable' atau 'net unreachable'. Dalam konteks keamanan, drop lebih disukai karena tidak memberikan informasi kepada penyerang.

TAHAP 8 (BONUS)

Monitoring Traffic dengan OVS Port Mirror



Tahap Bonus

Tahap ini bersifat opsional untuk mahasiswa yang sudah menyelesaikan Tahap 1-7 lebih awal.

OVS mendukung fitur port mirroring: menduplikasi traffic dari satu port ke port lain untuk keperluan monitoring tanpa mengganggu aliran traffic asli.

```
# Di Ubuntu-1: Tambahkan port monitoring ke OVS
sudo ip tuntap add tap-mirror mode tap
sudo ip link set tap-mirror up
sudo ovs-vsctl add-port ovs-br0 tap-mirror

# Aktifkan port mirroring: duplikasi semua traffic tap0 ke tap-mirror
sudo ovs-vsctl -- set bridge ovs-br0 mirrors=@m \
  -- --id=@tap0 get port tap0 \
  -- --id=@tap-mirror get port tap-mirror \
```



```
-- --id=@m create mirror name=mon0 \  
select-src-port=@tap0 select-dst-port=@tap0 \  
output-port=@tap-mirror  
  
# Monitor traffic menggunakan tcpdump pada interface mirror  
sudo tcpdump -i tap-mirror -n  
  
# Di tab terminal lain: generate traffic di tap0  
sudo tcpdump -i tap0 -n  
  
# Verifikasi mirror config  
sudo ovs-vsctl list mirror
```

C. Pertanyaan Analisis Praktikum 2



Jawab pertanyaan berikut berdasarkan hasil praktikum Anda

1. Jelaskan perbedaan mendasar antara Open vSwitch (OVS) dan Linux Bridge tradisional. Mengapa OVS disebut sebagai komponen SDN sementara Linux Bridge tidak?

2. Gambarkan diagram jaringan Docker yang Anda buat (bridge default dan virt-net). Jelaskan mengapa ct3 tidak bisa ping ct1 meskipun keduanya berjalan di host yang sama!

3. Apa perbedaan perilaku ping ketika firewall CHR menggunakan action=drop dibandingkan action=reject? Dalam skenario keamanan jaringan nyata, mana yang lebih Anda rekomendasikan dan mengapa?

4. Hubungkan ketiga konsep yang dipelajari (Virtual Networking, NFV, SDN) ke dalam sebuah narasi singkat: bagaimana ketiga konsep ini bekerja bersama dalam lingkungan cloud modern?

LEMBAR KERJA MAHASISWA

Nama Mahasiswa	
NIM	
Kelas / Kelompok	
Tanggal Praktikum	
Nilai	

A. Rekap Konfigurasi Jaringan (Isi Sesuai Hasil Lab)

Perangkat	Interface	IP Address / Konfigurasi
MikroTik CHR	ether1 (WAN/NAT)	
MikroTik CHR	ether2 (LAN-VIRT)	
MikroTik CHR	ether3 (LAN-VIRT-2)	
Ubuntu-1	Adapter 1	
Ubuntu-2	Adapter 1	
Docker ct1	eth0	
Docker ct2	eth0	
Docker ct3	eth0 (virt-net)	

B. Rekap Hasil Pengujian Konektivitas

Skenario	Hasil	Harapan	Sesuai?
Ubuntu-1 ping ke 192.168.10.1 (CHR)		Berhasil	
Ubuntu-1 ping ke 8.8.8.8 (internet)		Berhasil	
Ubuntu-2 ping ke 192.168.20.1 (CHR)		Berhasil	
Ubuntu-2 ping ke Ubuntu-1 (routing)		Berhasil	
ct1 ping ke ct2 (bridge default)		Berhasil	
ct3 ping ke ct1 (network berbeda)		Gagal	
Ubuntu-2 ping Ubuntu-1 + firewall drop		Gagal	

C. Checklist Penyelesaian Praktikum

✓	Item Penilaian
☐	Praktikum 1 Tahap 1: CHR berhasil diimport dan dapat login
☐	Praktikum 1 Tahap 2: IP ether2 terpasang, DHCP Server aktif, NAT Masquerade terkonfigurasi
☐	Praktikum 1 Tahap 3: Ubuntu-1 mendapat IP DHCP dan bisa ping ke internet
☐	Praktikum 1 Tahap 4: Ubuntu-2 berhasil ping ke Ubuntu-1 melalui routing CHR
☐	Praktikum 2 Tahap 5: OVS terinstall, bridge ovs-br0 terbuat, tap0 VLAN 10 dan tap1 VLAN 20 dikonfigurasi
☐	Praktikum 2 Tahap 6: Docker terinstall, ct1 dan ct2 dapat saling ping, ct3 terisolasi dari ct1/ct2
☐	Praktikum 2 Tahap 7: Firewall rule drop ICMP aktif, ping Ubuntu-2 ke Ubuntu-1 gagal, TCP masih bisa
☐	Semua pertanyaan analisis terjawab
☐	Lembar kerja terisi lengkap

REFERENSI

Sumber	Keterangan / URL
MikroTik CHR Documentation	https://wiki.mikrotik.com/wiki/Cloud_Hosted_Router
MikroTik IP Firewall Filter	https://wiki.mikrotik.com/wiki/IP/Firewall/Filter
Open vSwitch Documentation	https://docs.openvswitch.org/en/latest/
Docker Networking Guide	https://docs.docker.com/network/
VirtualBox User Manual	https://www.virtualbox.org/manual/UserManual.html
Ubuntu Server Guide 24.04	https://ubuntu.com/server/docs
OpenFlow Specification	https://opennetworking.org/sdn-resources/openflow/

Modul Praktikum Virtual Networking, NFV & SDN
S1 Teknik Informatika | Virtualisasi & Keamanan Jaringan